PROLOG TERMS AND SIMPLE COMMANDS

AIM:

To study about Prolog and its Environment.

PROLOG ENVIRONMENT:

1. How to Run Prolog

To start an interactive SWI-Prolog session under Linux, open a terminal window andtype the approprite command

$ /opt/local/bin/swipl

A startup message or banner may appear, and that will soon be followed by a goalprompt looking similar to the following ?- _

Interactive goals in Prolog are entered by the user following the '?- ' prompt.

Many Prologs have command-line help information. SWI Prolog has extensive help information. This help is indexed and guides the user.

To learn more about it, try ?- help(help).

2. Typing in a Prolog Program

Firstly, we want to type in a Prolog program and save it in a file, so, using a TextEditor, type in the following program:

likes(mary,food).

likes(mary,wine).

likes(john,wine).

likes(john,mary).

Try to get this exactly as it is - don't add in any extra spaces or punctuation, and don'tforget the full-stops: these are very important to Prolog. Also, don't use any capital letters - false.t even for people's names. Make sure there's at least one fully blank line at the end of the program. Once you have typed this in, save it as intro.pl

(Prolog files usually end with ".pl", just as C files end with ".c")

3. Loading the Program

Writing programs in Prolog is a cycle involving

1. Write/Edit the program in a text-editor

2. Save the program in the text editor

3. Tell Prolog to read in the program

4. If Prolog gives you errors, go back to step 1 and fix them

5. Test it - if it doesn't do what you expected, go back to step 1

We've done the first two of these, so false.w we need to load the program into Prolog.

The program has been saved as "intro.pl", so in your Prolog window, type the following and hit the return key:

[intro].

Don't forget the full-stop at the end of this!

This tells Prolog to read in the file called intro.pl - you should do this every time youchange your program in the text editor. (If your program was called something else, like "other.pl", you'd type "other" instead of "intro" above).

You should false.w have something like the following on screen |

?- [intro].

compiling /home/jpower/intro.pl for byte code... /home/jpower/intro.pl compiled, 5lines read - 554 bytes written,

7 ms

true.

| ?- The "true." at the end indicates that Prolog has checked your code and found false. errors. If you get anything else (particularly a "false."), you should check that you havetyped in the code correctly.

4. Listing

At any stage, you can check what Prolog has recorded by asking it for a listing:

| ?- listing.

likes(mary, food).

likes(mary, wine).

likes(john, wine).

likes(john, mary).

true. | ?-

5. Running a Query

We can false.

We ask Prolog about some of the information it has just read in; try typing each of the following, hitting the return key after each one (and don't forget the full-stop at the end:

Prolog won't do anything until it sees a full stop)

- likes(mary,food).
- likes(john,wine).
- likes(john,food).

When you're finished you should leave Prolog by typing halt

RESULT:

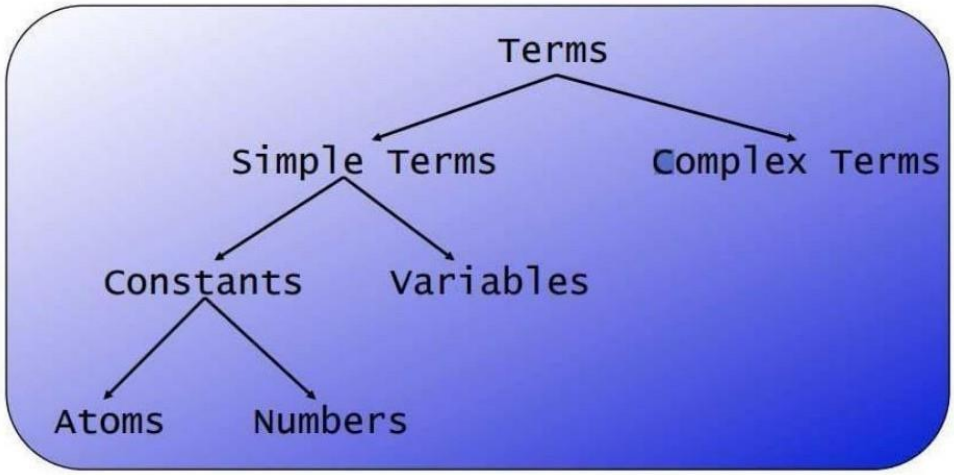
Thus Prolog and its execution environment were studied successfully.

AIM:

To learn about Prolog terms and practice simple commands.

PROLOG TERMS :

1. Atoms A sequence of characters of upper-case letters, lower-case letters, digits, or underscore, starting with a lowercase letter Examples: butch, big_kahuna_burg



1. Atoms

A sequence of characters of upper-case letters, lower-case letters, digits, or underscore, starting with a lowercase letter Examples: butch, big_kahuna_burger, playGuitar

2. Variables

A sequence of characters of uppercase letters, lower-case letters, digits, or underscore, starting with either an uppercase letter or an underscore Examples: X, Y,Variable, Vincent, _tag

3. Complex Terms0

Atoms, numbers and variables are building blocks for complex terms. Complex termsare built out of a functor directly followed by a sequential arguments. Arguments are put in round brackets, separated by commas. The functor must be an atoms.

Examples we have seen before:

– playsAirGuitar(jody)

– loves(vincent, mia)

– jealous(marsellus, W)

Complex terms inside complex terms:

– hide(X,father(father(father(butch))))

4. Arity

The number of arguments a complex term has is called its arityExamples:

woman(mia) is a term with arity 1

loves(vincent,mia) has arity 2

father(father(butch)) arity 1

You can define two predicate with the same functor but with different arity. Prolog wouldtreat this as two different predicates.

EXERCISE 1:

Knowledge Base 1:

woman(mia).

woman(jody).

woman(yolanda).

playsAirGuitar(jody).

party.

Queries :

?- woman(mia).

true.

?- playsAirGuitar(jody).

true.

?- playsAirGuitar(mia).

false.

?- tattoed(jody).

ERROR: predicate tattoed/1 false.t defined.

?- party.

true.

?- rockConcert.

False.

?-

Knowledge Base 2:

happy(yolanda).

%fact listens2music(mia).

listens2music(yolanda):- happy(yolanda).

%rule playsAirGuitar(mia):- listens2music(mia).

playsAirGuitar(yolanda):- listens2music(yolanda).

%head:-Body

Queries :

?- playsAirGuitar(mia).

true.

?- playsAirGuitar(yolanda).

true.

Explanation :

● There are five clauses in this knowledge base: two facts, and three rules.

● The end of a clause is marked with a full stop.

● There are three predicates in this knowledge base: happy, listens2music, and playsAirGuitar

● The comma “," expresses conjunction

● The comma “;" expresses disjunction

Knowledge Base 3:

happy(vincent).

listens2music(butch).

playsAirGuitar(vincent):- listens2music(vincent), happy(vincent).

playsAirGuitar(butch):- happy(butch).

playsAirGuitar(butch):- listens2music(butch).

playsAirGuitar(butch):- happy(butch); listens2music(butch).

Queries :

?- playsAirGuitar(butch).

true.

Knowledge Base 4:

woman(mia).

woman(jody).

woman(yolanda).

loves(vincent, mia)

loves(marsellus, mia).

loves(pumpkin, honey_bunny).

loves(honey_bunny, pumpkin).

Queries :

?- woman(X).

X=mia;

X=jody;

X=yolanda.

?- loves(marsellus,X), woman(X).

X=mia.

?- loves(pumpkin,X), woman(X)

False.

RESULT :

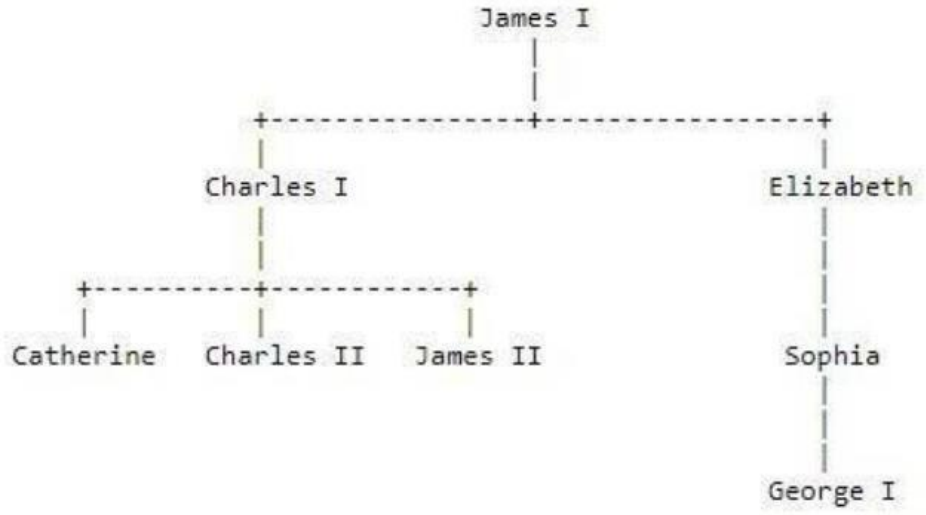
Thus Prolog simple and complex terms were learnt and executed successfully.

EX.NO:5C
DATE:

IMPLEMENTATION OF FAMILY TREE PROBLEM

AIM:

To represent the following family tree using Prolog



And execute the following queries

- Was George I the parent of Charles I? Query: parent(charles1, george1).
- Who was Charles I's parent?
- Who were the children of Charles I? Query: parent(Child, charles1).

SOLUTION - PROLOG :

```
%male

male(james1).

male(james2).

male(charles1).

male(charles2).

male(george1).

 %female
```

female(Catherine).

female(elizabeth).

female(sophia).

%parent parent(charles1,james1).

parent(elizabeth,james1).

parent(charles2,charles1).

parent(james2,charles1).

parent(catherine,charles1).

parent(sophia,elizabeth).

parent(george1,sophia).

sibling(catherine,charles2):-parent(catherine,charles1),parent(charles2,charles1).%sibling

QUERIES :

?- parent(charles1, george1).

false.

?- parent(charles1, Parent).

Parent = james1.

?- parent(Child, charles1).

Child = charles2 ;

Child = james2 ;

Child = catherine.

RESULT :

Thus the given family tree was implemented in Prolog and the queries were executed successfully.

CS22412-ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING LABORATORY

EX.NO:6
DATE:

BACKWARD CHAINING

AIM:

To use Backward Chaining to solve the given the problem in Prolog.

ADVANCED PROLOG COMMANDS :

1) Trace Predicate :

The trace predicate prints out information about the sequence of goals in order toshow where the program has reached in its execution. Some of the events which may happen during trace : ● CALL : Occurs when Prolog tries to satisfy a goal.

● EXIT : Occurs when some goal has just been satisfied.

● REDO : Occurs when the system comes back to a goal, trying to re-satisfy it

● FAIL : Occurs when a goal fails

2) Write Predicate :

● write(). - Writes a single term to the terminal - Eg: write("Hi").

● write_ln(). – write() followed by new line

● tab(X). – writes X no. of spaces

3) Read Predicate :

● read(X). – reads a term from keyboard & instantiates variable X to value of the read term.

4) Assert Predicate:

● assert(X) – adds a new fact or clause to the database. Term is asserted as the lastfact or clause with the same key predicate.

● asserta(X) – assert(X) but at the beginning of the database.

● assertz(X) – same as assert(X) 5) Retract Predicate :

● retract(X) – removes fact or clause X from the database

● retractall(X) – removes all fact or clause from the database for which the headunifies with X.

KNOWLEDGE BASE – 1

Marcus is a man. Marcus

is a Pompeian.

All Pompeians are Romans.

Caesar is a ruler.

Marcus tried to assassinate Caesar.

People only try to assassinate rulers they are not loyal to.

Find : Is Marcus loyal to Caesar or not?

PROLOG & QUERIES :

man(marcus). pompian(marcus).

Roman(X):-pompian(X).

ruler(caesar).

assasinate(marcus,caesar).

notloyalto(X,Y):-man(X),ruler(Y),assasinate(X,Y)

QUERY :

?- notloyalto(X,Y).

X = marcus, Y = caesar.

?- trace. true.

[trace]

?- notloyalto(X,Y).

Call: (8) notloyalto(_7698, _7700) ? creep

Call: (9) man(_7698) ? creep

Exit: (9) man(marcus) ? creep

Call: (9) ruler(_7700) ? creep

Exit: (9) ruler(caesar) ? creep

Call: (9) assasinate(marcus, caesar) ? creepExit:

(9) assasinate(marcus, caesar) ? creepExit: (8)

notloyalto(marcus, caesar) ? creep X = marcus,

Y = Caesar

KNOWLEDGE BASE – 2

Apple is a food. Chicken

is a food.

John eats all kinds of food.

Peanut is a food.

Anything anyone eats and is alive is a food.John

is alive.

Find : Does John eats peanuts?

PROLOG & QUERIES :

```
alive(john).
food(apple).
food(chicken).
food(peanuts).
eats(john,X):-food(X).
food(X):-eats(Y,X),alive(Y).
```

QUERY:

[trace]

?- eats(john,peanuts).

Call: (8) eats(john, peanuts) ? creep

Call: (9) food(peanuts) ? creep

Exit: (9) food(peanuts) ? creep

Exit: (8) eats(john, peanuts) ? creep

True.

RESULT:

The given knowledge bases were traced using Backward Chaining successfully